

CISP 400

Object Oriented Programming in C++

Dr. Fowler



F O L S O M L A K E C O L L E G E
EL DORADO CENTER | RANCHO CORDOVA CENTER

Last Time

- Class introduction, some C++ stuff.

Programming Exercise

Exercise

Programming Exercise

Write a C++ program that declares an array **Alpha** of 50 elements of type double. Initialize the array so that the first 25 elements are equal to the square of the index variable, in the last 25 elements are equal to three times the index variable. Output the array so that 10 elements per line are printed.

```

1 // pe0003.cpp
2 #include <iostream>
3 #include <cmath>
4 using namespace std;
5
6 int main() {
7     const int SIZE = 50;
8     const int MID = 25;
9     double alpha[SIZE] {0};
10
11     for (int i = 0; i < MID; i++) {
12         alpha[i] = i * i;
13     }
14
15     for (int i = MID; i < SIZE; i++)
16         alpha[i] = pow(i, 3.0);
17
18     const int MAXLINE = 10;
19     for (int i = 0; i < SIZE; i++) {
20         for (int j = 0; j < MAXLINE; j++)
21             cout << alpha[i + j] << " ";
22         cout << endl;
23         i += 9; // get the i counter right
24     }
25
26     return 0;
27 }

```

```

calebfowler@ubuntu:~/Desktop$ ./a.out
0 1 4 9 16 25 36 49 64 81
100 121 144 169 196 225 256 289 324 361
400 441 484 529 576 15625 17576 19683 21952 24389
27000 29791 32768 35937 39304 42875 46656 50653 54872 59319
64000 68921 74088 79507 85184 91125 97336 103823 110592 117649
calebfowler@ubuntu:~/Desktop$

```

```

1  // pe0003.cpp
2  #include <iostream>
3  #include <cmath>
4  using namespace std;
5
6  int main() {
7      const int SIZE = 50;
8      const int MID = 25;
9      double alpha[SIZE] {0};
10
11     for (int i = 0; i < MID; i++) {
12         alpha[i] = i * i;
13     }
14
15     for (int i = MID; i < SIZE; i++)
16         alpha[i] = pow(i, 3.0);
17
18     const int MAXLINE = 10;
19     for (int i = 0; i < SIZE; i++) {
20         for (int j = 0; j < MAXLINE; j++)
21             cout << alpha[i + j] << " ";
22         cout << endl;
23         i += 9; // get the i counter right
24     }
25
26     return 0;
27 }

```


Debrief

What worked about this exercise?

What could be improved about this exercise?

Why did I ask you to do

Connect activity with course objectives/SLO's.

Connect activity with course readings.

Chapter 10

Pointers

What's this chapter
about?

C++ Overview

- Review of pointers.

in CISP 360, you learned C++ has 3 types of variables ...

Simple - ints, floats, bools, etc.

Structured - struc's, classes, arrays, etc.

Pointers - pointers.



Big Idea

- It's easier to give someone your house address than a copy of your house.
- Necessary to build resizable data structures.

Related to Previous Chapters ...

- We are building progressively more complex data structures, so we need better ways of dealing with them.
- One of the primary (dis)advantages of C++.

Agenda

Ch 10 - Pointers

10.1 Pointers and the Address Operator

10.2 Pointer Variables

10.3 The Relationship between Arrays and pointers

10.4 Pointer Arithmetic

10.5 Initializing Pointers

10.6 Comparing Pointers

10.7 Pointers as Function Parameters

10.8 Pointers to Constants and Constant Pointers

10.9 Dynamic Memory Allocation

10.10 Returning Pointers from Functions

10.11 Pointers to Class Objects and Structures

10.12 Selecting Members of Objects

10.13 Smart Pointers

Agenda

Ch 10 - Pointers

10.1 Pointers and the Address Operator

10.2 Pointer Variables

10.3 The Relationship between Arrays and pointers

10.4 Pointer Arithmetic

10.5 Initializing Pointers

10.6 Comparing Pointers

10.7 Pointers as Function Parameters

10.8 Pointers to Constants and Constant Pointers

10.9 Dynamic Memory Allocation

10.10 Returning Pointers from Functions

10.11 Pointers to Class Objects and Structures

10.12 Selecting Members of Objects

10.13 Smart Pointers

10.1 Pointers and The Address Operator

```

1 // pointer demo
2
3 #include<iostream>
4 using namespace std;
5
6 int main() {
7     int x = 25;
8     int* ptr;           // ptr points to an int
9
10    ptr = &x;           // Store address of x in ptr
11    cout << "The value of x is " << x << endl;
12    cout << "The address of x is " << ptr << endl;
13    cout << "*ptr points to a value of " << *ptr << endl;
14    cout << "&x points to address " << &x << endl;
15
16    return 0;
17 }

```

Screen Shot 2016-09-11 at 6.48.34... Open with Preview

```

[Calebs-iMac:Desktop calebfowler$ ./a.out
The value of x is 25
The address of x is 0x7fff5f593ba8
*ptr points to a value of 25
&x points to address 0x7fff5f593ba8

```

```

1 // pointer demo
2
3 #include<iostream>
4 using namespace std;
5
6 int main() {
7     int x = 25;
8     int* ptr;    // ptr points to an int
9
10    ptr = &x;    // Store address of x in ptr
11    cout << "The value of x is " << x << endl;
12    cout << "The address of x is " << ptr << endl;
13    cout << "*ptr points to a value of " << *ptr << endl;
14    cout << "&x points to address " << &x << endl;
15
16    return 0;
17 }

```

Screen Shot 2016-09-11 at 6.48.34... Open with Preview

```

[Calebs-iMac:Desktop calebfowler$ ./a.out
The value of x is 25
The address of x is 0x7fff5f593ba8
*ptr points to a value of 25
&x points to address 0x7fff5f593ba8

```

Address-of operator

10.3 The Relationship between Pointers and Arrays

- Pointers really aren't all that scary.
- You've been using them for weeks now and you just didn't know it.
- An array variable is really a pointer!

```
//10-5.cpp -- Arrays & Pointers
#include <iostream>
using namespace std;

int main() {
    short numbers[] = {10, 20, 30, 40, 50};

    cout << "The first array element is "
         << *numbers << endl;

    return 0;
}
```

Run this, then try to access more than the first element using pointer notation.

C++ Operator Precedence

The following table lists the precedence and associativity of C++ operators. Operators are listed top to bottom, in descending precedence.

Precedence	Operator	Description	Associativity
1	::	Scope resolution	Left-to-right
2	a++ a-- type() type{} a() a[] . ->	Suffix/postfix increment and decrement Functional cast Function call Subscript Member access	
3	++a --a +a -a ! ~ (type) *a &a sizeof new new[] delete delete[]	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT C-style cast Indirection (dereference) Address-of Size-of ^[note 1] Dynamic memory allocation Dynamic memory deallocation	Right-to-left
4	.* ->*	Pointer-to-member	Left-to-right
5	a*b a/b a%b	Multiplication, division, and remainder	
6	a+b a-b	Addition and subtraction	
7	<< >>	Bitwise left shift and right shift	
8	<=>	Three-way comparison operator (since C++20)	
9	< <= > >=	For relational operators < and ≤ respectively For relational operators > and ≥ respectively	

https://en.cppreference.com/w/cpp/language/operator_precedence


```
1 // 10-6.cpp -- Use arrays with pointer notation
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     const int SIZE = 5;
8     int numbers[SIZE];
9
10    cout << "Enter " << SIZE << " numbers: ";
11    for(int count = 0; count < SIZE; count++)
12        cin >> *(numbers + count);
13    cout << numbers << endl;
14
15    cout << "Here are the inputs\n";
16    for (int count = 0; count < SIZE; count++)
17        cout << *(numbers + count) << " ";
18    cout << endl;
19
20    return 0;
21 }
```


10.5 Initializing Pointers

“Pointers may be initialized with the address of an existing object.”

–p.656

If we don't have data we want to point to (because we haven't declared it yet), we need a safe address to point to.

- We can initialize it to 0

```
int* pInt = 0;
```

```
double* pDoub = 0;
```

- We can also use a defined constant, **NULL**, to represent a pointer to 0.

```
int* pInt = NULL;
```

```
double* pDoub = NULL;
```

- C11 has built in keyword, `nullptr`, to initialize to 0.

```
int* pInt = nullptr;
```

```
double* pDoub = nullptr;
```

- Which means we can use 3 tests to see if a pointer is pointing to a valid memory location:

```
if (ptr != 0) {  
if (ptr != NULL) {  
if (ptr != nullptr) {
```

- OR you could just test to see if it's valid:

```
if (ptr) {  
    // do whatever you need to do with  
    // the pointer in here.  
}
```

Sidebar: Brace Initialization

- C11 has a new way of initializing variables - brace initialization.
- This automatically sets numeric types to zero and pointers to nullptr.

```
int myInt = 0;  
double myDoub = 0.0;  
int* pInt = nullptr;
```

```
int myInt{};  
double myDoub{};  
int* pInt{};
```


Why do you want to initialize your pointers?

Why do we need a safe address to point to?

Reliability

- Our programs are more reliable and easier to maintain when we KNOW what the computer is doing.

```
1  #include <iostream>
2  using namespace std;
3
4  ✓ int main() {
5      int* pInt;
6
7      pInt = new int;
8
9      cout << *pInt << endl;    // >0 runs fine
10
11     return 0;
12 }
```

```
1  #include <iostream>
2  using namespace std;
3
4  ✓ int main() {
5      int* pInt;
6
7      pInt = new int;
8
9      cout << *pInt << endl;    // >0 runs fine
10
11     return 0;
12 }
```

The program runs fine.

Related issue: Where is address 0 and do we want to read and write to it?

- Let's go ahead and initialize this the correct way ...

```
1 // example10-5.cpp -- §10.5 lecture example
2 #include <iostream>
3 using namespace std;
4
5 ✓ int main() {
6     int* pInt;
7
8     pInt = nullptr;
9
10    cout << *pInt << endl;
11
12    return 0;
13 }
```

```
1 // example10-5.cpp -- §10.5 lecture example
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int* pInt;
7
8     pInt = nullptr;
9
10    cout << *pInt << endl;
11
12    return 0;
13 }
```

```
> ./main
signal: segmentation fault (core dumped)
>
```

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int* pInt;
6
7      pInt = new int;
8
9      cout << *pInt << endl;    // >0 runs fine
10
11     return 0;
12 }

```

Logic Error

```

1  // example10-5.cpp -- §10.5 lecture example
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      int* pInt;
7
8      pInt = nullptr;
9
10     cout << *pInt << endl;
11
12     return 0;
13 }

```

Syntax Error

Tradeoff between a logic error and a syntax error.


```
1 // example10-5.cpp -- §10.5 lecture example
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int* pInt;
7
8     pInt = nullptr;
9
10    if (pInt) cout << *pInt << endl;
11    else cout << "Pointer error!" << endl;
12
13    return 0;
14 }
```

```
➤ ./main
Pointer error!
➤
```



```
1 // example10-5.cpp -- §10.5 lecture example
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int* pInt;
7
8     pInt = nullptr;
9     pInt = new int{};
10    *pInt = 10;
11
12    if (pInt) cout << *pInt << endl;
13    else cout << "Pointer error!" << endl;
14
15    return 0;
16 }
```

➤ ./main

10

➤ □

Note

- This setup will not tell us if the memory location is valid.
- We could delete the memory location and the program would still run. The If statement would NOT pick that up unless we reset the pointer to null.

Hw 1 Preflight

Exercise

Exercise Goals

- Go over the homework and check understanding.
- Get questions answered.
- Discuss potential designs.

Task

- Read the next homework due in Canvas.
- Write down any questions you have about it.

Task - II

- Pair Up
- Go over each other's questions & see if you can answer them.
- about 3 minutes ...



5

Task - III

- Ask me any unanswered questions.



What did we do
today?

C++ Overview

- Review of pointers.

Agenda

Ch 10 - Pointers

10.1 Pointers and the Address Operator

10.2 Pointer Variables

10.3 The Relationship between Arrays and pointers

10.4 Pointer Arithmetic

10.5 Initializing Pointers

10.6 Comparing Pointers

10.7 Pointers as Function Parameters

10.8 Pointers to Constants and Constant Pointers

10.9 Dynamic Memory Allocation

10.10 Returning Pointers from Functions

10.11 Pointers to Class Objects and Structures

10.12 Selecting Members of Objects

10.13 Smart Pointers



NEXT TIME

Agenda

Ch 10 - Pointers

10.1 Pointers and the Address Operator

10.2 Pointer Variables

10.3 The Relationship between Arrays and pointers

10.4 Pointer Arithmetic

10.5 Initializing Pointers

10.6 Comparing Pointers

10.7 Pointers as Function Parameters

10. 8 Pointers to Constants and Constant Pointers

10.9 Dynamic Memory Allocation

10.10 Returning Pointers from Functions

10.11 Pointers to Class Objects and Structures

10.12 Selecting Members of Objects

10.13 Smart Pointers



Problem: Pointers allow access to anything (security risk). Do we have a tool which gives us the advantages of pointers without the security risk?

Yes!

Reference Variables

- A reference variable allows us to create a link to the **original** variable.
- The links are good as long as the original variable is in scope.
- Think of it like an alias.

Comparison between Pointers and Reference Variables

Both provide direct access to a variable.

Contrast between Pointers and Reference Variables

Pointer variables access value by dereferencing.
Reference variables don't need to do that.

Pointers can be reassigned to other memory locations.
References cannot be changed.

Pointers can have multiple levels of indirection.
References only have one level of indirection.

Pointers can be assigned to null.
References cannot be assigned to null.

- The advantages and disadvantages mean that there are some things we need pointers to do - we can't do those things with reference variables.

```
1 //6-26(mod).cpp -- Reference Variable Demo (modified)
2 // from C++ Brief p.351
3 #include <iostream>
4 using namespace std;
5
6 // Function Prototypes
7 void doubleNum(int &);
8 void getNum(int &);
9
10 int main() {
11     int value;
12     getNum(value);
13     doubleNum(value);
14     cout << "Doubling that number is " << value << endl;
15
16     return 0;
17 }
18
19 void getNum(int& userNum) {
20     cout << "Enter a number: ";
21     cin >> userNum;
22 }
23
24 void doubleNum(int& refVar) {
25     refVar *= 2;
26 }
```

Debug12-11.cpp

Exercise

Exercise Goals

- Debugging Practice

Debug Exercise

- Correct the following code.
- We'll spend about 10 minutes on this, then we'll go over it.
- You do not need to turn this in.


```
1 // debug12.11.cpp
2 // Find the area and perimeter of a rectangle.
3
4 #include <iostream>
5 using namespace std;
6
7 int main() {
8     double* length;
9     double* width;
10
11     cout << fixed << showpoint << setprecision(2);
12
13     length = new double;
14     length = 6.5;
15     &width = 3.0;
16
17     cout << "Area: " << (*length) * (&width) << ", ";
18     cout << "Perimeter: " << 2 * (*length + width) << endl;
19
20     return 0;
21 }
```



```
1 // debug12.11.Fix.cpp -- Corrected code
2 // Find the area and perimeter of a rectangle.
3
4 #include <iostream>
5 #include <iomanip>
6 using namespace std;
7
8 int main() {
9     double* length;
10    double* width;
11
12    cout << fixed << showpoint << setprecision(2);
13
14    length = new double;
15    *length = 6.5;
16    width = new double;
17    *width = 3.0;
18
19    cout << "Area: " << (*length) * (*width) << ", ";
20    cout << "Perimeter: " << 2 * (*length + *width) << endl;
21
22    return 0;
23 }
```

Desk Check

Exercise

Exercise Goals

- Review of code syntax
- Check understanding of syntax

Desk Check Exercise

- Pair up
- What's the output of the following code?
DO NOT TYPE THIS IN AND EXECUTE IT.
- We'll spend about 10 minutes on this, then go over it.

```

1 // output12.13.cpp
2 // What's the output of this code?
3
4 #include <iostream>
5 #include <iomanip>
6 using namespace std;
7
8 int main() {
9     double* trip1Cost = new double;
10    double* trip2Cost = new double;
11    double* trip3Cost = new double;
12    double* max;
13
14    *trip1Cost = 100.00;
15    *trip2Cost = 350.00;
16    trip3Cost = trip2Cost;
17    trip2Cost = trip1Cost;
18    trip1Cost = new double;
19    *trip1Cost = 175;
20    *trip3Cost = 275;
21
22    cout << fixed << showpoint << setprecision(2);
23    cout << "Trip total cost: $"
24         << (*trip1Cost + *trip2Cost + *trip3Cost) << endl;
25
26    max = trip1Cost;
27    if (*max < *trip2Cost)
28        max = trip2Cost;
29    if (*max < *trip3Cost)
30        max = trip3Cost;
31
32    cout << "Hightest trip cost: $" << *max << endl;
33
34    return 0;
35 }

```

```

1 // output12.13.cpp
2 // What's the output of this code?
3
4 #include <iostream>
5 #include <iomanip>
6 using namespace std;
7
8 int main() {
9     double* trip1Cost = new double;
10    double* trip2Cost = new double;
11    double* trip3Cost = new double;
12    double* max;
13
14    *trip1Cost = 100.00;
15    *trip2Cost = 350.00;
16    trip3Cost = trip2Cost;
17    trip2Cost = trip1Cost;
18    trip1Cost = new double;
19    *trip1Cost = 175;
20    *trip3Cost = 275;
21
22    cout << fixed << showpoint << setprecision(2);
23    cout << "Trip total cost: $"
24         << (*trip1Cost + *trip2Cost + *trip3Cost) << endl;
25
26    max = trip1Cost;
27    if (*max < *trip2Cost)
28        max = trip2Cost;
29    if (*max < *trip3Cost)
30        max = trip3Cost;
31
32    cout << "Highest trip cost: $" << *max << endl;
33
34    return 0;
35 }

```

```

gcc version 4.6.3
>
Trip total cost: $550.00
Highest trip cost: $275.00
>

```

References

- [GW17] Gaddis, Tony. Starting out with C++. Early objects. Starting out with C++. Early objects: Pearson, 2017.
- [Me01] Meyers, Scott. Effective STL: 50 Specific Ways to Improve Your Use of the Standard Template Library. Addison-Wesley Professional, 2001.